

Computing Longest Previous non-overlapping Factors

Maxime Crochemore^{a,b}, German Tischler^{a,*},¹

^a King's College London, London, UK

^b Université Paris-Est, France

ARTICLE INFO

Article history:

Received 20 January 2010

Received in revised form 25 November 2010

Accepted 1 December 2010

Available online 7 December 2010

Communicated by F.Y.L. Chin

Keywords:

Design of algorithms

Longest previous factor

Suffix array

Ziv–Lempel factorisation

Text compression

Detection of repetitions

ABSTRACT

The Longest Previous non-overlapping Factor table (LPnF) stores for each position of a string the maximal length of factors occurring both there and in the preceding part of the string. The notion is a slight variant of the LPF table described before and used for text compression. The LPnF table is an essential element for the design of efficient algorithms on strings as it is related to a certain type of Ziv–Lempel factorisation used for this purpose. We show how to compute the LPnF table in linear time from the suffix array of the string when it is drawn from an integer alphabet. The algorithm is a non-immediate extension of the LPF computation and it does not require any other sophisticated data structure than the suffix array of the input string.

© 2010 Elsevier B.V. All rights reserved.

1. Longest previous non-overlapping factors

We consider a string y of length n on the alphabet A : $y = y[0..n-1]$. The problem is to compute the Longest Previous non-overlapping Factor table LPnF defined, for $0 \leq i < n$, by

$$\text{LPnF}[i] = \max\{k \mid y[i..i+k-1] \text{ occurs in } y[0..i-1]\}.$$

The problem may be regarded as an extension of a factorisation of strings, called the f -factorisation (see [3]) used to design algorithms on strings. The factorisation is similar to the Ziv–Lempel factorisation [12] used for text compression but is less powerful than it when considered for this purpose (see [1]).

The f -factorisation plays an important role in String Algorithms. The intuitive reason is that, when processing a string on-line, the work that has to be done on an element of the factorisation can usually be skipped because it has already been done on one of its previous occur-

rences. A typical application of this idea resides in algorithms to compute repetitions in strings (see [2,11,10]). For example, the algorithm in [10] reports all maximal repetitions (called runs) occurring in a string in $O(n \log |A|)$ time. It runs in linear time if a Suffix Array is used in place of a Suffix Tree [4]. Indeed the technique seems to be the only technique that leads to linear-time algorithms independently of the alphabet size for this type of question. Using a balanced divide-and-conquer technique leads to $O(n \log n)$ running time algorithms. Instead, although the f -factorisation implements an unbalanced divide-and-conquer method, it yields $O(n \log |A|)$ time solutions, or even linear time solutions with a light hypothesis on the alphabet.

The LPnF table allows further improvements on dedicated string algorithms and text compression methods although its computation cost is not higher than the f -factorisation.

The main data structure used in the article is the Suffix Array of the input text y . It provides an ideal data structure to solve many questions requiring an index on all the factors of a string. The structure can be built in linear time by different methods for sorting the suffixes of the text (see [3]) possibly adding the algorithm of [9] to compute

* Corresponding author.

E-mail addresses: maxime.crochemore@kcl.ac.uk (M. Crochemore), german.tischler@kcl.ac.uk (G. Tischler).

¹ Newton Fellow at King's College London.

the Longest Common Prefix table (see also [8]). The result holds, if the text is on an integer alphabet, that is, if the alphabet of the text can be sorted in linear time.

The notion of Longest Previous non-overlapping Factors is a variant of that of Longest Previous Factor, where the previous occurrences of a factor can overlap the current occurrence. This latter notion has been introduced by Franek et al. as part of their concept of a Quasi Suffix Array (their π array is the LPF table) in [7], where the authors presented a direct computation running in $O(n \log n)$ average time. A linear-time computation from the Suffix Array of the text is described in [5]. The solution requires only a fixed amount of memory space in addition to the Suffix Array and the output table.

A direct computation of the LPnF table using a linear-time string-matching algorithm provides an effective quadratic-time solution on many inputs. In this article we extend the solution in [5]. Our algorithm (Section 4) still runs in linear time but has one more level of complexity despite the fact that LPnF and LPF are quite similar tables. In contrast to the solution presented in [6] the algorithm in this paper is based on a deeper analysis of the difficulties of LPnF computation over LPF computation. Therefore it does not require any other sophisticated data structures like indexes for efficiently solving the range minimum query problem.

2. Definitions

Let $y = y[0..n-1]$ be a word of length n over the finite ordered alphabet A . Let SUF be the array of lexicographically sorted suffixes positions on y , i.e.

$$y[SUF[0]..n-1] < \dots < y[SUF[n-1]..n-1].$$

Let $\text{lcp}(r, q)$ denote the length of the longest common prefix of the strings $y[SUF[r]..n-1]$ and $y[SUF[q]..n-1]$ and let the array LCP be given by $LCP[r] = \text{lcp}(r-1, r)$, for $r = 1, \dots, n-1$, and $LCP[0] = 0$. The pair of arrays (SUF, LCP) is commonly known as the *suffix array* of y . For example, the suffix array of the string `bbbbabbbbabbabbab` is as shown in Table 1 (see also Fig. 1).

If A is an integer alphabet in the interval $[0, n^c]$ for some constant c , the suffix array can be computed in linear time $O(n)$ (cf. [3]). For general alphabets the suffix array can be computed in $O(n \log n)$ time.

Using the suffix array of y , for $r < t$, $\text{lcp}(r, t)$ can be computed with the formula

$$\text{lcp}(r, t) = \min\{LCP[r+1], \dots, LCP[t]\}.$$

Table 1

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$SUF[i]$	14	11	8	4	15	13	10	7	3	12	9	6	2	5	1	0
$LCP[i]$	0	2	5	3	0	1	3	6	4	1	4	7	5	2	6	3

Table 2

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$y[i]$	b	b	b	b	a	b	b	b	a	b	b	a	b	b	a	b
$LPF[i]$	0	3	2	1	0	6	5	4	3	7	6	5	4	3	2	1
$LPnF[i]$	0	1	2	1	0	4	4	4	3	5	4	3	4	3	2	1

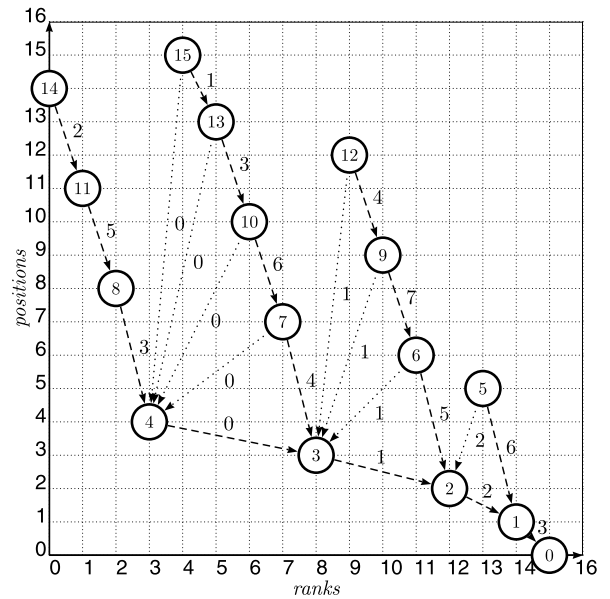


Fig. 1. Graph for the word `bbbbabbbbabbabbab`. The vertex for rank r is at position $(r, SUF[r])$ and is labelled by $SUF[r]$. If $\text{next}[r] \neq n$, the nodes $SUF[r]$ to $SUF[\text{next}[r]]$ are linked by a dashed edge labelled by the value $LPF_{>}[r]$. Analogously, the arrays prev and $LPF_{<}$ are shown using dotted edges.

This property implies that if we approach some rank r from below (resp. above), i.e. following a sequence of ranks $s[i] = s + i$ for $s < r$ and $i = 0, 1, \dots, r - s$, (resp. $s[i] = s - i$ for $s > r$ and $i = 0, 1, \dots, s - r$) the value $\text{lcp}(r, s[i])$ increases monotonically with i .

The Longest Previous Factor table for y is defined for any position i , $0 \leq i < n$, by

$$LPF[i]$$

$$= \max\{k \mid y[i..i+k-1] \text{ occurs at a position } j < i\},$$

(1)

i.e. for each position i it stores the length of the longest factor occurring previously in y . For example, the text $y = \text{bbbbabbbbabbabbab}$ has the following LPF and LPnF tables (Table 2).

For each rank r the value $LPF[SUF[r]]$ can be computed by finding the largest rank $r_{<}$ such that $r_{<} < r$ and $SUF[r_{<}] < SUF[r]$, the smallest rank $r_{>}$ such that $r_{>} > r$ and $SUF[r_{>}] < SUF[r]$ and choosing the maximum of $\text{lcp}(r_{<}, r)$ and $\text{lcp}(r_{>}, r)$. If $r_{<}$ does not exist, we set

$r_{<} = -1$ and $\text{lcp}(-1, r) = 0$. Likewise, if $r_{>}$ does not exist, we set $r_{>} = n$ and $\text{lcp}(r, n) = 0$.

On ranks of suffixes positions we define both the arrays prev and next by $\text{prev}[r] = r_{<}$ and $\text{next}[r] = r_{>}$ and the arrays $\text{LPF}_{>}$ and $\text{LPF}_{<}$ by $\text{LPF}_{>}[r] = \text{lcp}(r, \text{next}[r])$ and $\text{LPF}_{<}[r] = \text{lcp}(\text{prev}[r], r)$ for $r = 0, \dots, n-1$. It is clear that the array LPF can be written as

$$\text{LPF}[\text{SUF}[r]] = \max\{\text{LPF}_{<}[r], \text{LPF}_{>}[r]\}. \quad (2)$$

The arrays SUF , prev , next , $\text{LPF}_{>}$ and $\text{LPF}_{<}$ of a word can be displayed as shown in Fig. 1 for the word `bbbbabb-babbabbab`.

If $r' > r$ and $\text{SUF}[r'] > \text{SUF}[r]$ for all r'' such that $r < r' \leq r'$, then $r_{>} \geq r'_{>}$. Symmetrically, if $r' < r$ and $\text{SUF}[r'] > \text{SUF}[r]$ for all r'' such that $r' \leq r'' < r$, then $r_{<} \leq r'_{<}$. Using these observations, the arrays prev , next , $\text{LPF}_{>}$ and $\text{LPF}_{<}$ can all be computed in linear-time $O(n)$ (cf. [5]).

3. Computing Longest Previous non-overlapping Factors

The LPnF table can be computed in quadratic time by the naïve algorithm LPnF_QUADRATIC . The function $\text{KMP}(u, v)$ returns the length of the longest prefix of the string u which is a factor of the string v . It can be implemented to run in time $O(|u| + |v|)$ using the Knuth–Morris–Pratt algorithm (see [3]).

$\text{LPnF_QUADRATIC}(y, n)$

```

1 for  $i \leftarrow 0$  to  $n-1$  do
2    $\text{LPnF}[i] \leftarrow \text{KMP}(y[i..n-1], y[0..i-1])$ 
3 return  $\text{LPnF}$ 
```

Let us consider the computation of the value $\text{LPnF}[\text{SUF}[r]]$ for some rank r using the Suffix Array. Without loss of generality assume that a longest previous non-overlapping factor for rank r exists at rank $r' > r$. We may encounter two cases. If $\text{lcp}(r, \text{next}[r])$ is not larger than the difference $d(r) = \text{SUF}[r] - \text{SUF}[\text{next}[r]]$ of the respective positions, then the longest previous factor at position $\text{SUF}[\text{next}[r]]$ does not overlap the occurrence at position $\text{SUF}[r]$. Therefore we have $\text{LPnF}[\text{SUF}[r]] = \text{LPF}[\text{SUF}[r]] = \text{LPF}_{>}[r]$. Instead, if $\text{lcp}(r, \text{next}[r]) > d(r)$, then we may have $\text{LPnF}[r] < \text{LPF}[r]$, as the longest previous factor at position $\text{SUF}[\text{next}[r]]$ overlaps its occurrence at $\text{SUF}[r]$. But the factor may occur at a smaller position as well, where the overlap is smaller.

Let $\zeta(r, q) = \min\{\text{lcp}(r, q), \text{SUF}[r] - \text{SUF}[q]\}$ for two ranks r and q satisfying $\text{SUF}[q] < \text{SUF}[r]$. In analogy to the array $\text{LPF}_{>}$ we define the array $\text{LPnF}_{>}$ by

$$\text{LPnF}_{>}[r] = \max\{0\} \cup \{k \mid k = \zeta(r, q) \text{ s.t. } q > r \text{ and } \text{SUF}[q] < \text{SUF}[r]\}$$

for $r = 0, \dots, n-1$. The array $\text{LPnF}_{<}$ is defined likewise. The LPnF array can be computed in linear time from the arrays $\text{LPnF}_{<}$ and $\text{LPnF}_{>}$. In the following we focus on the computation of the array $\text{LPnF}_{>}$ because the computation of $\text{LPnF}_{<}$ can be performed analogously. Let $\text{next}^0[r] = r$ and $\text{next}^i[r] = \text{next}[\text{next}^{i-1}[r]]$ for an integer $i > 0$, where we assume $\text{next}[n] = n$.

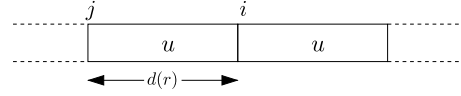


Fig. 2. A repetition of period $|u|$ in the word y : $i = \text{SUF}[r]$, $j = \text{SUF}[\text{next}[r]]$ and $|u| = d(r) = \text{SUF}[r] - \text{SUF}[\text{next}[r]]$.

Clearly $d(r, i) = \text{SUF}[r] - \text{SUF}[\text{next}^i[r]]$ increases monotonically with i (where we assume $\text{SUF}[n] = -1$) and $l(r, i) = \text{lcp}(r, \text{next}^i[r])$ decreases monotonically with i , where $l(r, n) = 0$. Let $\lambda(r, i) = \zeta(r, \text{next}^i[r])$. Therefore, as a consequence of the definitions and the discussion we get

$$\text{LPnF}_{>}[r] = \max_{i=1,2,\dots,n} \lambda(r, i). \quad (3)$$

Thus we can compute the value $\text{LPnF}_{>}[r]$ for each rank r by successively evaluating the function $\lambda(r, i)$ for increasing values of i until the maximum has been reached. Each single evaluation of λ can be performed in constant time based on information accumulated during the previous evaluations. Unfortunately, the total number of such evaluations over all ranks can be quadratic in n as it can be checked for input words a^n , where a is a letter.

4. Linear-time computation of the LPnF table

We now consider in detail the situations provoking a quadratic number of evaluations of the λ function in the algorithm outlined above and change the algorithm so they can be avoided.

Whenever we evaluate λ more than once for the computation of $\text{LPnF}_{>}[r]$ for some rank r , this is indeed created by a repetition in the input word y and can be depicted as shown in Fig. 2. The repetition has an exponent of at least 2. In addition, the period $p = d(r)$ of the repetition is minimal. We can describe the situation by the decomposition $y = vuuw$ where the position of the first occurrence of u is $\text{SUF}[\text{next}[r]]$ and the position of the second is $\text{SUF}[r]$. Assume that p were not minimal (i.e. u is not primitive). Then we would have $u = u'^k$ for some non-empty primitive word u' and some integer $k > 1$. Consequently we could write $y = vu'^k u'^k w$. Then we would have

$$uw = u'^k w < u'^{k+1} w < \dots < u'^{2k} w = uuw$$

which would imply a different next array. Thus p is minimal.

We introduce the definition of *chains* to describe the sequence of ranks considered by evaluations of the λ function during the computation of $\text{LPnF}_{>}$ for one rank. The definitions and proofs we give in the following all treat the next , $\text{LPF}_{>}$ complex only, since the argumentation for prev , $\text{LPF}_{<}$ is analogous.

Definition 1. A chain is a sequence of ranks $r_0, \dots, r_k$ satisfying

1. $k > 0$
2. $0 \leq r_0 < \dots < r_k < n$
3. $r_{i+1} = \text{next}[r_i]$ for $i = 0, \dots, k-1$
4. $\text{SUF}[r_i] - \text{SUF}[r_{i+1}] < \text{LPF}_{>}[r_i]$ for $i = 0, \dots, k-1$

A chain $r_0, \dots, r_k$ of constant period p is a chain such that $\text{SUF}[r_i] - \text{SUF}[r_{i+1}] = \text{SUF}[r_0] - \text{SUF}[r_1] = p$ for $i = 0, \dots, k-1$. A maximal chain $r_0, \dots, r_k$ of constant period p is an inextensible chain of constant period p , that is, neither $r_0, \dots, r_k, \text{next}[r_k]$, nor $r', r_0, \dots, r_k$ for $r' < r_0$ are chains of constant period p . The values c_0 and o_0 for a chain $r_0, \dots, r_k$ of constant period p are defined by $c_0 = \lfloor \text{LPF}_{>}[r_0]/p \rfloor$ and $o_0 = \text{LPF}_{>}[r_0] - c_0 p$.

The graph in Fig. 1 contains the maximal chains of constant period (1, 2), (6, 7), (9, 10, 11), (11, 12), (12, 14, 15) and (13, 14).

As seen above, chains are induced by repetitions in the input word. The next lemmas state two more properties of maximal chains of constant period.

Lemma 1. Let $r_0, \dots, r_k$ be a maximal chain of constant period p . Then the equality $\text{LPF}_{>}[r_i] = \text{LPF}_{>}[r_0] + ip = (c_0 + i)p + o_0$ holds for $i = 0, \dots, k-1$. In particular $\text{LPF}_{>}[r_0] < \text{LPF}_{>}[r_1] < \dots < \text{LPF}_{>}[r_{k-1}]$.

Proof. Let v be the factor of length $\text{LPF}_{>}[r_0]$ starting at position $\text{SUF}[r_0]$. Then v also occurs at position $\text{SUF}[r_1]$ and is the longest factor with this property by definition of $\text{LPF}_{>}$. Let $u = y[\text{SUF}[r_1].. \text{SUF}[r_0] - 1]$ be the factor of length p occurring at $\text{SUF}[r_1]$. Since the chain has constant period, u also occurs at position $\text{SUF}[r_2]$. Therefore uv occurs at both positions $\text{SUF}[r_1]$ and $\text{SUF}[r_2]$, is the longest factor with this property, and then $\text{LPF}_{>}[r_1] = |uv| = \text{LPF}_{>}[r_0] + p$. Iterating the same argument for other values of i leads to the conclusion. $\square$

Lemma 2. Let $r_0, \dots, r_k$ be a maximal chain of constant period p . If the inequality $\text{next}[r_k] \neq n$ holds, $\text{SUF}[r_i] - \text{SUF}[\text{next}[r_k]] \geq \text{lcp}(r_i, \text{next}[r_k])$ for each $i = 0, \dots, k-1$.

Proof. As $\text{SUF}[r_i] - \text{SUF}[\text{next}[r_k]]$ falls monotonically and $\text{lcp}(r_i, \text{next}[r_k])$ grows monotonically with i for $i = 0, \dots, k-1$, it is sufficient to prove the equation

$$\text{SUF}[r_{k-1}] - \text{SUF}[\text{next}[r_k]] \geq \text{lcp}(r_{k-1}, \text{next}[r_k]),$$

where $\text{lcp}(r_{k-1}, \text{next}[r_k]) = \min\{\text{LPF}_{>}[r_{k-1}], \text{LPF}_{>}[r_k]\}$. We consider two cases.

1. $\text{SUF}[r_k] - \text{SUF}[\text{next}[r_k]] \geq \text{LPF}_{>}[r_k]$: trivial, as the considered factor at position $\text{SUF}[\text{next}[r_k]]$ does not overlap position $\text{SUF}[r_k]$.
2. $\text{SUF}[r_k] - \text{SUF}[\text{next}[r_k]] < \text{LPF}_{>}[r_k]$: rank r_k marks the beginning of a chain of period $q = \text{SUF}[r_k] - \text{SUF}[\text{next}[r_k]] \neq p$. This situation can be depicted in the following way:

$\text{SUF}[\text{next}[r_k]]$	$\text{SUF}[r_k]$	$\text{SUF}[r_{k-1}]$
$a_1 a_2 \dots a_q$	$a_1 a_2 \dots a_p$	$a_1 a_2 \dots a_p$

We consider two sub-cases:

- (a) $q > p$. Then we have the inequality $\text{LPF}_{>}[r_k] < 2p$ as otherwise there would be a repetition of period $p < q$ starting at $\text{SUF}[\text{next}[r_k]]$. This implies the inequality $\min\{\text{LPF}_{>}[r_{k-1}], \text{LPF}_{>}[r_k]\} \leq 2p$ and in consequence also $p + q \geq \min\{\text{LPF}_{>}[r_{k-1}], \text{LPF}_{>}[r_k]\}$.

- (b) $q < p$. Then we have the inequality $\text{LPF}_{>}[r_k] < 2q$ as otherwise there would be a repetition of period $q < p$ starting at $\text{SUF}[r_k]$. This implies the inequality $\min\{\text{LPF}_{>}[r_{k-1}], \text{LPF}_{>}[r_k]\} \leq 2q$ and in consequence also $p + q \geq \min\{\text{LPF}_{>}[r_{k-1}], \text{LPF}_{>}[r_k]\}$.

This gives the conclusion and ends the proof. $\square$

The lemmas provide two important hints. The first lemma tells us that while computing the $\text{LPnF}_{>}$ value for rank r_i , the algorithm outlined in Section 3 will follow the chain of next pointers while evaluating the function λ up to the point where it either reaches the end of the chain or up to the smallest j such that

$$\begin{aligned} \text{SUF}[r_i] - \text{SUF}[r_j] &\geq \text{lcp}(r_i, r_j) \\ \Leftrightarrow (j - i)p &\geq (c_0 + i)p + o_0 \\ \Leftrightarrow j &\geq (c_0 + 2i) + \lceil o_0/p \rceil. \end{aligned} \quad (4)$$

As long as the value of j does not lie beyond the end of the chain (i.e. $j \leq k$), the value $\text{LPnF}_{>}[r_i]$ equals the value $\text{LPF}_{>}[r_i]$. The first i for which this no longer necessarily holds is thus obtained via

$$(c_0 + 2i) + \lceil o_0/p \rceil > k \Leftrightarrow i > \left\lfloor \frac{k - (c_0 + \lceil o_0/p \rceil)}{2} \right\rfloor. \quad (5)$$

If i is beyond this limit, we have two cases to consider. First, if $\text{next}[r_k] = n$, then $\text{LPnF}_{>}[r_i]$ is given by $\min\{\text{LPF}_{>}[r_i], \text{SUF}[r_k] - \text{SUF}[r_i]\} = \min\{(c_0 + i)p + o_0, (k - i)p\} = (k - i)p$, as a longest non-overlapping previous factor starts at position $\text{SUF}[r_k]$. Second, if $\text{next}[r_k] \neq n$, the second lemma tells us, that we can always obtain the $\text{LPnF}_{>}$ value by considering the $\text{LPF}_{>}$ values on the chain and one $\text{LPF}_{>}$ value beyond the chain (the one at rank r_k). Then a longest previous non-overlapping factor may start at either position $\text{SUF}[r_k]$ (with length $(k - i)p$) or at position $\text{SUF}[\text{next}[r_k]]$ (with length $\min\{\text{SUF}[r_i] - \text{SUF}[\text{next}[r_k]], \min\{\text{LPF}_{>}[r_i], \text{LPF}_{>}[r_k]\}\} = \min\{\text{LPF}_{>}[r_i], \text{LPF}_{>}[r_k]\}$). Thus in this case, $\text{LPnF}_{>}[r_i] = \max\{(k - i)p, \min\{\text{LPF}_{>}[r_i], \text{LPF}_{>}[r_k]\}\}$.

We use these hints to design the algorithm LPnF_NEXT_LINEAR below. The first loop initialises the $\text{LPnF}_{>}$ array to the $\text{LPF}_{>}$ array. The second loop handles maximal chains. Starting ranks of maximal chains are detected by checking from low to high ranks for unhandled ranks r such that $\text{next}[r] \neq n$ and $d(r) < \text{LPF}_{>}[r]$. When the beginning of a maximal chain of constant period is detected, the algorithm computes the length k and the final rank r_k of the chain in the first while loop. The second while loop assigns the $\text{LPnF}_{>}$ values to the ranks on the chain according to the discussion above.

The argumentation above shows that LPnF_NEXT_LINEAR computes the $\text{LPnF}_{>}$ array. Let us consider its runtime. The first for loop takes linear time. The second loop handles all the maximal chains of constant period in the input. Assume that a chain $r_0, r_1, \dots, r_{k-1}, r_k$ is treated. Then both while loops consider rank r_i , for $0 \leq i < k$, exactly once and the second loop marks the rank as handled. So the work for each rank is amortised to constant time and the overall worst-case runtime of the algorithm is linear.

LPnF_NEXT_LINEAR(SUF, next, LPF_>, n)

```

1  for  $r \leftarrow 0$  to  $n - 1$  do
2    LPnF>[ $r$ ]  $\leftarrow$  LPF>[ $r$ ]
3  for  $r \leftarrow 0$  to  $n - 1$  do
4    if rank  $r$  is unhandled and next[ $r$ ]  $\neq n$  then
5      ( $p, l_0$ )  $\leftarrow$  (d( $r$ ), LPF>[ $r$ ])
6      if  $p < l_0$  then
7        ▷ handle chain starting at  $r_0 = r$ 
8         $c_0 \leftarrow \lfloor l_0/p \rfloor$ 
9         $o_0 \leftarrow l_0 - c_0 p$ 
10       ▷ compute length of chain  $k$  and rank  $r_k$ 
11       ( $k, r_k$ )  $\leftarrow$  (0,  $r$ )
12       while next[ $r_k$ ]  $\neq n$  and d( $r_k$ ) < LPF>[ $r_k$ ] and
13         d( $r_k$ ) =  $p$  do
14         ( $k, r_k$ )  $\leftarrow$  ( $k + 1$ , next[ $r_k$ ])
15         ▷ Fill in LPnF> values
16          $t \leftarrow \lfloor \frac{k - (c_0 + \lfloor o_0/p \rfloor)}{2} \rfloor$ 
17         ( $i, r_i$ )  $\leftarrow$  (0,  $r$ )
18         while  $r_i \neq r_k$  do
19           if  $i > t$  then
20             LPnF>[ $r_i$ ]  $\leftarrow$  ( $k - i$ ) $p$ 
21             if next[ $r_k$ ]  $\neq n$  then
22                $l_{r_i r_k} \leftarrow \min\{\text{LPF}_{>}[r_i], \text{LPF}_{>}[r_k]\}$ 
23               LPnF>[ $r_i$ ]  $\leftarrow \max\{\text{LPnF}_{>}[r_i], l_{r_i r_k}\}$ 
24               mark rank  $r_i$  as handled
25               ( $i, r_i$ )  $\leftarrow$  ( $i + 1$ , next[ $r_i$ ])
26       return LPnF>
```

Theorem 1. The algorithm LPnF_NEXT_LINEAR computes the array LPnF_> in time $O(n)$.

Corollary 1. The Longest Previous non-overlapping Factor table of a string of length n can be computed in time $O(n)$.

The C++ implementations of the algorithms described in this paper are available at <http://www.dcs.kcl.ac.uk/staff/tischler/src/lpnf.zip>.

References

- [1] J. Berstel, A. Savelli, Crochemore factorization of Sturmian and other infinite words, in: R. Kralovic, P. Urzyczyn (Eds.), MFCS 2006, in: LNCS, vol. 4162, Springer, 2006, pp. 157–166.
- [2] M. Crochemore, Transducers and repetitions, Theoretical Computer Science 45 (1) (1986) 63–86.
- [3] M. Crochemore, C. Hancart, T. Lecroq, Algorithms on Strings, Cambridge University Press, Cambridge, UK, 2007, p. 392.
- [4] M. Crochemore, L. Ilie, Computing longest previous factor in linear time and applications, Inf. Process. Lett. 106 (2) (2008) 75–80.
- [5] M. Crochemore, L. Ilie, C. Iliopoulos, M. Kubica, W. Rytter, T. Waleń, LPF computation revisited, in: D. Fronček, J. Kratochvíl, M. Miller (Eds.), IWOC 2009, in: LNCS, vol. 5874, Springer, 2009, pp. 158–169.
- [6] M. Crochemore, C. Iliopoulos, M. Kubica, W. Rytter, T. Waleń, Efficient algorithms for two extensions of the LPF table: the power of Suffix Arrays, in: J. van Leeuwen, A. Muscholl, D. Peleg, J. Pokorný, B. Rumpé (Eds.), SOFSEM 2010, in: LNCS, vol. 5901, Springer, Berlin, 2010, pp. 296–307.
- [7] F. Franek, J. Holub, W.F. Smyth, X. Xiao, Computing quasi suffix arrays, Journal of Automata, Languages and Combinatorics 8 (4) (2003) 593–606.
- [8] J. Kärkkäinen, G. Manzini, S.J. Puglisi, Permuted longest-common-prefix array, in: G. Kucherov, E. Ukkonen (Eds.), CPM, in: LNCS, vol. 5577, Springer, 2009, pp. 181–192.
- [9] T. Kasai, G. Lee, H. Arimura, S. Arikawa, K. Park, Linear-time longest-common-prefix computation in suffix arrays and its applications, in: CPM 2001, in: LNCS, vol. 2089, Springer, 2001, pp. 181–192.
- [10] R.M. Kolpakov, G. Kucherov, Finding maximal repetitions in a word in linear time, in: Foundations of Computer Science, 1999, pp. 596–604.
- [11] M.G. Main, Detecting leftmost maximal periodicities, Discrete Appl. Math. 25 (1989) 145–153.
- [12] J. Ziv, A. Lempel, A universal algorithm for sequential data compression, IEEE Transactions on Information Theory 23 (3) (1977) 337–343.